

Neural Networks: What They Are, How They Work, and How to Tune Them

Prepared for NotebookLM Study Use

March 27, 2026

Contents

1 Why Neural Networks Exist

Neural networks are function approximators. Their job is to learn a mapping from inputs x to outputs y by adjusting a large number of parameters. In practice, this means:

- for images, they learn to map pixels to labels;
- for text, they learn to map token sequences to probabilities or generated text;
- for tabular data, they learn to map feature columns to scores or probabilities;
- for ranking or recommendation, they learn to map user-item information to relevance.

The core idea is simple: instead of hand-writing a large set of rules, we define a flexible parameterized function and let data determine the parameter values.

At a high level, a neural network is built from layers. Each layer transforms a vector into another vector. By composing many such transformations, the network can learn highly non-linear relationships.

2 The Basic Unit: A Single Neuron

The simplest building block is the artificial neuron:

$$z = w^\top x + b$$

$$a = \phi(z)$$

where:

- x is the input vector,
- w is the weight vector,
- b is the bias,
- z is the pre-activation,
- ϕ is a non-linear activation function,
- a is the neuron's output.

Without the activation function, stacking layers would collapse into one big linear function. Non-linearity is therefore essential. It is what allows the model to learn thresholds, interactions, curved decision boundaries, and hierarchical representations.

2.1 Intuition

Think of $w^\top x + b$ as a learned scoring rule. The neuron measures how strongly the input aligns with a pattern. The activation then transforms that score into something more useful for the next layer.

Example: if a model is learning credit risk, one neuron might implicitly detect “high balance plus repeated contact plus certain job/education combination”. Another neuron may detect “client recently contacted and previous outcome was successful”.

3 From Neurons to Layers and Networks

A dense layer applies many neurons in parallel:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$a^{(1)} = \phi(z^{(1)})$$

Then the next layer takes $a^{(1)}$ as input:

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

$$a^{(2)} = \phi(z^{(2)})$$

Eventually we produce an output:

$$\hat{y} = f_{\theta}(x)$$

where θ denotes all parameters in the network.

3.1 Why Depth Helps

Depth allows representation reuse. Early layers can detect low-level patterns, middle layers can combine them into richer concepts, and later layers can convert these concepts into decisions. In tabular settings, depth often helps by learning interactions that are cumbersome to engineer manually.

4 Forward Pass

The forward pass is the act of taking an input, passing it through every layer, and producing a prediction.

For binary classification, the last layer usually produces a logit:

$$\ell = W^{(L)}a^{(L-1)} + b^{(L)}$$

Then the sigmoid converts it to a probability:

$$\hat{p} = \sigma(\ell) = \frac{1}{1 + e^{-\ell}}$$

$$\hat{p} \in (0, 1)$$

This probability can be interpreted as the model's belief that the class is positive.

5 Loss Functions

The loss function tells the model what it means to be wrong.

5.1 Mean Squared Error

For regression:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

5.2 Binary Cross-Entropy

For binary classification:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

Cross-entropy strongly penalizes confident wrong predictions. That is usually exactly what we want.

5.3 What the Loss Is Really Doing

The model does not directly optimize accuracy, AUC, or F1 in most setups. Instead, it optimizes a differentiable surrogate such as cross-entropy. The hope is that minimizing this training objective produces better downstream metrics.

6 Backpropagation

Backpropagation is the procedure that computes gradients of the loss with respect to each parameter.

The key fact is the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \cdot \frac{\partial a^{(L)}}{\partial a^{(L-1)}} \cdots \frac{\partial a^{(1)}}{\partial W^{(1)}}$$

Conceptually:

- the forward pass computes outputs;
- the loss measures error;
- backpropagation distributes responsibility for that error backward through the network;
- the optimizer then updates the parameters using those gradients.

6.1 Why It Works

Each parameter is nudged in the direction that most reduces the loss, at least locally. Over many batches and epochs, the network gradually moves into a region of parameter space that yields useful predictions.

7 Gradient Descent and Optimizers

The simplest update is gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

where η is the learning rate.

7.1 Mini-batch Training

In modern deep learning, gradients are usually computed on mini-batches rather than the full dataset:

- full-batch is stable but slow;
- stochastic updates are noisy;
- mini-batches are the practical compromise.

7.2 Common Optimizers

SGD: simple and sometimes strong, especially with careful schedules.

Momentum: smooths updates by accumulating velocity.

Adam: adapts per-parameter learning rates and is the default choice in many projects.

AdamW: Adam with decoupled weight decay; often preferred in modern architectures.

7.3 Learning Rate Is Usually the Most Important Hyperparameter

If the learning rate is too high:

- loss oscillates,
- training diverges,
- validation metrics are unstable.

If it is too low:

- training is painfully slow,
- the model may plateau early,
- you waste epochs without learning meaningful structure.

8 Activation Functions

8.1 ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Advantages:

- simple,
- fast,
- mitigates some vanishing-gradient issues.

Risk:

- “dead” neurons if activations stay negative.

8.2 Leaky ReLU and GELU

Leaky ReLU preserves a small negative slope. GELU is common in transformers because it behaves more smoothly and often works well in deeper models.

8.3 Sigmoid and Tanh

Historically important, but in hidden layers they saturate easily and can cause vanishing gradients. They are more common in output layers or specific recurrent settings than in modern deep feed-forward stacks.

9 Initialization

Bad initialization can break training before optimization really begins.

9.1 Why Initialization Matters

If activations or gradients explode, optimization becomes unstable. If they vanish, the network barely learns.

9.2 Common Schemes

- Xavier/Glorot initialization: often used with tanh-like activations.
- He/Kaiming initialization: often used with ReLU-like activations.

The underlying goal is to keep the scale of activations and gradients in a healthy range across layers.

10 Regularization

Neural networks are flexible enough to memorize. Regularization helps them generalize.

10.1 L2 Weight Decay

Adds a penalty on large weights. This discourages overly sharp or fragile solutions.

10.2 Dropout

Randomly zeros part of the hidden representation during training. This prevents excessive co-adaptation and acts like a form of ensemble averaging.

10.3 Early Stopping

Stop training when validation performance stops improving. This is one of the most practical forms of regularization.

10.4 Data Augmentation

Critical in images and useful in text. In tabular data it is trickier and must be used carefully, because random perturbations can easily destroy semantics.

11 Normalization

11.1 Feature Scaling

For many neural networks, input scale matters a lot. If one feature ranges from 0 to 1 and another from 0 to 100000, optimization becomes harder.

Common choices:

- standardization,
- log transforms for skewed positive features,
- clipping extreme outliers,
- embedding layers for categorical features rather than naive integer coding.

11.2 Batch Normalization and Layer Normalization

Batch normalization normalizes activations using batch statistics and is common in CNNs and MLPs. Layer normalization normalizes across features within each example and is standard in transformers.

12 The Typical Training Loop

The full training process usually looks like this:

1. prepare and split the data;
2. preprocess features;
3. initialize model parameters;
4. run forward pass on a batch;
5. compute loss;
6. backpropagate gradients;
7. update parameters with an optimizer;
8. evaluate on validation data;
9. repeat for many epochs;
10. stop when the model stops improving or begins to overfit.

13 How to Read Training Curves

Training and validation curves are one of the fastest ways to diagnose model behavior.

13.1 Underfitting

Signs:

- training loss remains high,
- validation loss remains high,
- both metrics are poor.

Potential fixes:

- larger model,
- more training,

- better features,
- better optimization,
- less regularization.

13.2 Overfitting

Signs:

- training loss keeps improving,
- validation loss worsens,
- validation metric peaks early then declines.

Potential fixes:

- stronger regularization,
- earlier stopping,
- more data,
- smaller model,
- better validation design.

14 The Most Important Hyperparameters to Tune

There are many hyperparameters, but not all deserve equal attention.

14.1 Tier 1: Usually Highest Impact

Hyperparameter	Why it matters
Learning rate	Controls update size; the most common source of failure.
Model width/depth	Determines representational capacity.
Batch size	Affects optimization noise, throughput, and sometimes generalization.
Regularization strength	Controls overfitting.
Training duration / epochs	Too short underfits; too long may overfit.

14.2 Tier 2: Important but Context-Dependent

Hyperparameter	Typical role
Dropout rate	Helps regularize, but too much can cripple learning.
Weight decay	Smoothly constrains parameter magnitude.
Activation choice	ReLU, GELU, Leaky ReLU can behave differently.
Scheduler	Cosine decay, step decay, warmup can stabilize training.
Initialization	Often implicit, but sometimes decisive in deep models.

15 A Practical Tuning Strategy

The biggest tuning mistake is changing too many things at once. A disciplined process is better:

1. establish a simple baseline that trains reliably;
2. verify that data splits and metrics are correct;
3. tune learning rate first;
4. then tune capacity;
5. then tune regularization;
6. finally tune schedules and secondary details.

15.1 Step 1: Get a Stable Baseline

Before searching for the best model, make sure the baseline:

- trains without divergence,
- improves on validation,
- reproduces across seeds,
- saves metrics and checkpoints cleanly.

15.2 Step 2: Tune Learning Rate

Pick a reasonable architecture and sweep only learning rate. If nothing learns, architecture is not yet the main problem. Optimization is.

15.3 Step 3: Tune Capacity

Once optimization is stable:

- increase width if the model is too weak,
- increase depth if interactions are too complex,
- reduce size if the model memorizes too easily.

15.4 Step 4: Tune Regularization

If the training metric is much better than validation:

- add or increase dropout,
- increase weight decay,
- use earlier stopping,
- reduce model size.

15.5 Step 5: Tune Seeds and Ensembling

If one run is good but unstable, try:

- multiple random seeds,
- checkpoint averaging,
- prediction ensembling.

Neural networks often benefit materially from seed averaging.

16 Common Failure Modes

16.1 Divergence

Symptoms:

- exploding loss,
- NaNs,
- wild validation swings.

Possible causes:

- learning rate too high,
- bad preprocessing,
- unstable initialization,
- numerical issues from extreme values.

16.2 No Learning

Symptoms:

- loss barely changes,
- predictions stay near the prior,
- gradients may be tiny.

Possible causes:

- learning rate too low,
- poor feature scaling,
- saturated activations,
- a data pipeline bug.

16.3 Validation Is Good but Leaderboard Is Bad

This often means:

- the validation split is not matching the test distribution,
- the model is exploiting fold-specific artifacts,
- calibration differs between validation and test,
- higher local metric does not necessarily transfer to leaderboard.

17 Neural Networks on Tabular Data

Tabular data is special. Tree models are often very strong, especially on medium-sized datasets with heterogeneous feature types. Neural networks can still help, but they usually require more care:

- careful preprocessing,
- useful categorical handling,
- stronger regularization,
- multiple seeds,
- patience about the fact that a locally stronger NN may still transfer differently than a tree model.

17.1 Typical Tabular NN Design Choices

- numeric features: standardize or log-transform where appropriate;
- categorical features: embeddings or clean one-hot encoding;
- hidden stack: MLP, residual MLP, gated MLP, or attention-based tabular architecture;
- loss: binary cross-entropy for binary classification;
- metric tracking: AUC if the competition is AUC-based.

17.2 Why Tabular NNs Often Work Best as Diversity Models

Even when a tree ensemble beats the neural network on local validation, the neural network may still contribute useful prediction diversity. If two models make different mistakes, a blend can outperform both.

18 From MLPs to Transformers

A transformer is still a neural network, but it replaces simple fixed mixing of features with learned attention.

In a standard MLP, every layer mixes all coordinates through a dense matrix. In a transformer, representations are updated by attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The model learns which inputs should influence which other inputs.

18.1 Why Transformers Matter

Transformers became dominant because attention is a powerful and flexible way to model relationships. In language, each token can attend to other relevant tokens. In tabular transformers, columns or feature tokens can interact through the same general mechanism.

18.2 What Changes When Tuning Transformers

Compared with simple MLPs, transformers introduce additional important hyperparameters:

- embedding dimension,
- number of layers,
- number of attention heads,
- feed-forward width,
- dropout in attention and MLP blocks,
- warmup schedule,
- weight decay and normalization details.

Transformers are usually more sensitive to optimization details than shallow MLPs.

19 A Tuning Checklist

When tuning a neural network, ask these questions in order:

1. Is the data pipeline correct?
2. Is the validation split trustworthy?
3. Do features have sane scales?
4. Does the model learn at all?
5. Is learning rate appropriate?
6. Is the model too small or too large?
7. Is regularization too weak or too strong?
8. Are results stable across seeds?
9. Does calibration matter for the downstream objective?
10. Does the model add diversity relative to the current best system?

20 What Strong Practical Tuning Looks Like

Good neural network tuning is not blind search. It is an iterative engineering loop:

1. build a reproducible baseline;
2. observe failures carefully;
3. change one hypothesis at a time;
4. keep logs, seeds, and artifacts;
5. compare not only raw accuracy but also diversity, stability, and transfer behavior.

This matters because the best local neural network is not always the best system component. In many real projects, the winning role of a neural network is not to replace the incumbent model, but to complement it.

21 Conclusion

Neural networks are parameterized functions trained by gradient-based optimization. Their power comes from composition, non-linearity, and representation learning. Their difficulty comes from the fact that training quality depends on many interacting choices: preprocessing, architecture, optimizer, schedule, regularization, validation design, and randomness.

The most useful mental model is this:

- architecture controls what can be represented;
- optimization controls whether the model can find a good solution;
- regularization controls whether that solution generalizes;
- validation controls whether you can trust what you are seeing.

If you tune in that order, you will usually move faster and make better decisions.